

MEMOIRE

Mémoire de validation entreprise – 2e année

Module Académique ETNA : PRO-ENT5

Problématique d'Ingénierie :

« Comment garantir la cohérence transactionnelle et tarifaire d'un moteur de réservation aérien agrégeant des protocoles hybrides (GDS et NDC) ? »

Candidat :

Jérémy SOLER

Apprenant Expert 2e Année

Spécialité : Architecture Logicielle &
Systèmes Distribués

Poste : Apprenti — Architecte Systèmes
d'Information

Encadrement :

Tuteur Entreprise :

Reouven SAMAMA

Fondateur & CEO — Everbloo

Tuteur Pédagogique :

Comité de Suivi des Mémoires — ETNA

Période d'immersion professionnelle : Octobre 2023 – Novembre 2026 (38 mois)

Année académique 2025 – 2026

Remerciements

La réalisation de ce mémoire d'ingénierie et l'aboutissement de ces trente-huit mois d'immersion professionnelle au cœur de la TravelTech sont le fruit d'un investissement soutenu, rendu possible grâce à l'accompagnement, la confiance et l'expertise de nombreuses personnes que je tiens chaleureusement à remercier.

Mes premiers remerciements s'adressent tout particulièrement à mon tuteur d'entreprise, **Reouven SAMAMA**, Fondateur, CEO et Tech Lead d'**Everbloo**, pour son accueil au sein de l'entreprise, sa vision stratégique et la liberté d'initiative totale qu'il m'a accordée sur les choix d'architecture logicielle de la plateforme **Metis**. Ses conseils avisés, son exigence technique et sa confiance continue m'ont permis de franchir un cap décisif dans ma posture d'architecte systèmes d'information.

J'exprime ma profonde gratitude aux fondateurs et dirigeants de **Metis Digital**, **Jérôme BENBIHI**, **Bernard MOLLE** et **David MARCIANO**, pour la richesse de leurs retours métier et leur expertise pointue des enjeux de la distribution aérienne internationale.

Je tiens également à remercier chaleureusement l'équipe produit d'Everbloo, notamment nos Product Owners **Steeven ALIEL**, **Brenda SECK** et **Véronique DANG**, pour la clarté des spécifications fonctionnelles et la fluidité de nos synergies au quotidien.

Mes vifs remerciements vont à mes pairs et collègues du pôle développement, tout particulièrement **Jiong Jiong LIN**, ainsi qu'à l'ensemble de nos équipes d'ingénieurs réparties entre la France et la Tunisie. La richesse de nos échanges techniques, la rigueur collective lors des revues de code et l'esprit d'émulation permanente ont constitué un moteur déterminant pour surmonter les défis complexes inhérents à l'interopérabilité des protocoles aériens.

Je remercie l'équipe pédagogique et les intervenants de l'**ETNA** (École des Technologies Numériques Avancées). L'excellence du cursus d'Expert en Ingénierie Informatique et la qualité du suivi pédagogique ont été des atouts précieux pour structurer ma démarche scientifique et réflexive.

Enfin, j'adresse mes plus tendres remerciements à ma famille, à mes proches et à mes camarades de promotion pour leur soutien indéfectible, leur écoute et leurs encouragements tout au long de ce parcours d'exception.

Résumé

Au cours d’une période d’immersion professionnelle de trente-huit mois (octobre 2023 – novembre 2026) au sein de la société **SAS Everbloo**, mon travail d’ingénierie logicielle a porté sur la conception architecturale, l’optimisation des performances et le développement full-stack de la plateforme **Metis**. Cet écosystème opère en tant que moteur de distribution aérienne B2B et outil de réservation d’entreprise (*Self-Booking Tool* – SBT) pour des réseaux d’agences de voyages (notamment TourCom) et des gestionnaires de déplacements professionnels. En tant qu’Apprenti Architecte Systèmes d’Information, je suis intervenu sur les quatre dépôts du monorepo (`api-aerial`, `front-reservation`, `front-dashboard` et `api-dashboard`), représentant 1 554 commits portant sur la modularité des services, l’interfaçage de fournisseurs tiers et la fiabilisation des flux de réservation sous forte charge.

Problématique d’ingénierie :

La distribution de billets d’avion combine aujourd’hui deux modèles techniques opposés : d’une part, les systèmes de distribution globale historiques (GDS : Sabre, Amadeus), reposant sur des sessions d’état synchrones et le protocole EDIFACT ; d’autre part, la norme IATA NDC (*New Distribution Capability*), fondée sur des API REST/JSON et XML sans état délivrant des offres temporaires. Dans cet environnement hétérogène, la question centrale traitée dans ce mémoire est : « *Comment garantir la cohérence transactionnelle et tarifaire au sein d’un moteur de réservation de billets d’avion agréant des protocoles hybrides (GDS et NDC) ?* »

Démarche méthodologique et réalisations :

Pour répondre à cette problématique sans dégrader l’expérience utilisateur ni surcharger les serveurs, j’ai mis en place une architecture orientée services (SOA) articulée autour d’un backend Node.js/Express (avec l’ORM Sequelize) et de frontends réactifs Nuxt 3 / Vue 3 (Pinia, TypeScript, Vuetify 3). Mes réalisations techniques majeures comprennent :

- La création d’une couche d’adaptateurs canoniques (`SabreAdapter.ts`) traduisant les charges utiles hétérogènes vers un modèle pivot commun ;
- Le développement d’un intercepteur HTTP pour le renouvellement silencieux des jetons OAuth2 avec file d’attente FIFO (*request queuing*) pour absorber l’expiration des sessions sans interruption ;
- L’élaboration d’un algorithme déterministe de réconciliation passagers (`matchesPaxRefId`) gérant les cas limites (nourrissons sans siège rattachés à un adulte, noms composés, nettoyage de services orphelins) ;
- L’implémentation d’un flux de retarification réactif (*reshopping*) qui intercepte les variations tarifaires avant paiement (HTTP 409) et permet une confirmation interactive sans perte de saisie ;
- La refonte asynchrone du pipeline de conversion multi-devises respectant la norme ISO 4217 et le découplage des segments passifs entre Sabre et Amadeus.

Résultats et validation en production :

Sur un volume de code maintenu supérieur à 192 000 lignes nettes, ces réalisations ont été validées en environnement réel. D'après les journaux structurés Winston et le suivi d'erreurs Sentry, le taux d'échec lors de la création d'ordre NDC est tombé sous la barre des 0,1 % (hors pannes matérielles côté compagnies). Le temps moyen d'exécution des requêtes de shopping a été réduit de 28 % (de 4,2 s à 3,0 s sur un échantillon de 10 000 requêtes), et aucun avis de débit (*Debit Memo*) lié à des segments passifs dupliqués n'a été constaté sur plus de deux ans d'exploitation.

Bilan réflexif :

Cette expérience m'a permis de confronter les patrons d'architecture logicielle aux aléas concrets de la production (décalages de fuseaux horaires, micro-écarts d'arrondis monétaires, pannes intermittentes d'API tierces). Elle retrace également mon évolution personnelle : l'apprentissage de la gestion du stress en production, la négociation technique avec les Product Owners et l'adoption d'une communication factuelle et transparente auprès des agences partenaires.

Mots-clés :

Distribution Aérienne, GDS (Sabre, Amadeus), IATA NDC, Architecture SOA, Micro-Frontends, Nuxt 3, Vue 3, TypeScript, Node.js, Express, Cohérence Transactionnelle, Reshopping, OAuth2 Silent Renewal, Multi-Devises ISO 4217, Segments Passifs, Tests Mocha/Chai.

Abstract

Over a thirty-eight-month professional apprenticeship (October 2023 to November 2026) at **SAS Everbloo**, my software engineering work focused on the architectural design, performance optimization, and full-stack development of the **Metis** platform. Metis is a B2B airline distribution engine and Self-Booking Tool (SBT) used by travel agency networks (notably TourCom) and corporate travel managers. As Information Systems Architect Apprentice, I contributed across all four repositories of the monorepo (**api-aerial**, **front-reservation**, **front-dashboard**, and **api-dashboard**), authoring 1,554 commits centered on service modularity, third-party provider interfacing, and transaction resilience under high traffic.

Engineering Problem Statement:

Airline distribution currently bridges two conflicting technical paradigms: legacy Global Distribution Systems (GDS: Sabre, Amadeus) relying on stateful, synchronous EDIFACT sessions, and the IATA New Distribution Capability (NDC) standard based on stateless REST/JSON and XML APIs serving short-lived offers. In this heterogeneous environment, the central question addressed in this thesis is: « *How to guarantee transactional and pricing consistency in an airline booking engine aggregating hybrid protocols (GDS and NDC)?* »

Methodology & Technical Realizations:

To resolve protocol disparities and pricing volatility without sacrificing user experience, a Service-Oriented Architecture (SOA) was implemented, coupling a Node.js/Express backend (Sequelize ORM) with reactive Nuxt 3 / Vue 3 frontends (Pinia, TypeScript, Vuetify 3). Key technical achievements include:

- Designing a canonical Adapter layer (**SabreAdapter.ts**) normalizing heterogeneous payloads into a shared domain model;
- Implementing an HTTP interceptor for silent OAuth2 token renewal with FIFO request queuing to handle session expirations without service interruptions;
- Developing a deterministic passenger matching algorithm (**matchesPaxRefId**) handling complex edge cases (lap infants linked to adults, compound surnames, and orphaned ancillary cleanup);
- Engineering a reactive reshopping flow that intercepts pre-checkout price drifts (HTTP 409) and allows interactive confirmation without resetting form state;
- Building an asynchronous multi-currency pipeline compliant with ISO 4217 and decoupling passive segment generation between Sabre and Amadeus.

Production Validation & Results:

Maintained across more than 192,000 net lines of code, these solutions were validated in real-world production. Monitoring via structured Winston logs and Sentry error tracking demonstrated that NDC order creation failure rates dropped below 0.1% (excluding provider

hardware downtime). Average shopping response times decreased by 28% (from 4.2 s to 3.0 s over a 10,000-query benchmark), and zero passive segment debit memos (ADMs) were incurred over two years of live operations.

Reflective Analysis:

This experience grounded software design theory in the gritty realities of production (timezone discrepancies, currency rounding nuances, and intermittent third-party API failures). It also highlights my personal growth: managing production incident stress, leading technical negotiations with Product Owners, and establishing transparent, fact-based communication with B2B agency partners.

Keywords:

Airline Distribution, GDS (Sabre, Amadeus), IATA NDC, SOA Architecture, Micro-Frontends, Nuxt 3, Vue 3, TypeScript, Node.js, Express, Transactional Consistency, Reshopping, OAuth2 Silent Renewal, ISO 4217 Multi-Currency, Passive Segments, Mocha/Chai Testing.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iv
1 Introduction Générale et Cadrage Stratégique	1
1.1 Contexte Sectoriel et Évolution de la Distribution Aérienne	1
1.2 Énonciation de la Problématique d'Ingénierie	1
1.3 Plan et Organisation du Mémoire	2
I Contexte Professionnel, Écosystème Métier de l'Aérien	3
2 Présentation Organisationnelle et Rôle d'Architecte Systèmes d'Information	4
2.1 L'Entreprise d'Accueil : SAS Everbloo	4
2.2 La Plateforme Metis : Enjeux et Écosystème Partenaire	4
2.3 Organisation de l'Équipe et Quotidien de Développement	5
2.3.1 Positionnement et Périmètre d'Intervention	5
3 Écosystème Technologique et Architecture de la Plateforme Metis	7
3.1 Stack Technologique et Environnement de Développement	7
3.2 Architecture Modulaire et Découpage des Services	7
3.3 Transition vers la Problématique	8
4 Formulation et Contextualisation de la Problématique Centrale	9
4.1 Énonciation du Défi d'Ingénierie	9
4.2 Formulation des Quatre Hypothèses de Travail	9
4.2.1 Hypothèse 1 : Abstraction Canonique via le Patron Adaptateur	10
4.2.2 Hypothèse 2 : Réconciliation Déterministe des Passagers et Pipeline Multi-Devises	10
4.2.3 Hypothèse 3 : Moteur Réactif de Retarification avec Validation Interactive	10
4.2.4 Hypothèse 4 : Découplage des Points de Vente et Maîtrise des Segments Passifs	10
II Réalisations Techniques, Résolution de la Problématique	11
5 Intégration Hybride des Protocoles GDS et IATA NDC	12
5.1 Expérience Utilisateur et Moteur de Recherche Metis Connect	12
5.2 Couche d'Abstraction Canonique via le Patron Adaptateur	12
5.3 Terminal et Commandes Cryptiques pour Billettistes	15
5.4 Gestion Découplée des Segments Passifs Amadeus et Sabre	15
5.5 Synchronisation des Dossiers Voyageurs (PNR)	16

6	Cycle Transactionnel de Réservation et Retarification Temps Réel	17
6.1	Détection et Gestion des Écarts Tarifaires (Reshopping)	17
6.2	Algorithme Déterministe de Réconciliation des Passagers	18
7	Optimisation du Moteur de Recherche et Filtrage Multi-Compagnies	21
7.1	Composant de Filtrage Avancé et Persistance Locale	21
7.2	Optimisation des Requêtes de Shopping Serveur	21
8	Architecture Financière et Traitement Asynchrone Multi-Devises	22
8.1	Résolution Asynchrone de la Devise Point de Vente	22
8.2	Formatage Dynamique et Norme ISO 4217	22
9	Back-Office d'Administration et Gouvernance des Agences	23
9.1	Interface de Paramétrage des Points de Vente	23
9.2	Synchronisation par Lots des Profils Sabre	23
10	Industrialisation, Tests Automatisés et Résultats Mesurés	24
10.1	Couverture de Tests Unitaires et d'Intégration	24
10.2	Synthèse Factuelle des Résultats en Production	24
III	Démarche Réflexive, Analyse Critique et Maturation Professionnelle	25
11	Réponse Démonstrative à la Problématique et Validation des Hypothèses	26
11.1	Bilan de la Démarche d'Ingénierie	26
11.2	Validation et Nuances des Hypothèses de Travail	26
12	Analyse Critique, Galères de Production et Défis Techniques Réels	28
12.1	Gestion du Cycle de Vie OAuth2 et File d'Attente de Requêtes	28
12.2	Retour d'Expérience sur Trois Incidents Majeurs de Production	30
12.2.1	1. L'Écart de Centime sur la Taxe de Solidarité (Taxe Chirac)	30
12.2.2	2. Les Timestamps sans Fuseau Horaire chez Aegean et Turkish Airlines	31
12.2.3	3. Interblocage (Deadlock SQL) sous Sequelize lors de Réservations Simultanées	31
12.3	Dette Technique et Choix d'Architectures Alternatives	31
13	Maturation Humaine, Posture Professionnelle et Réalités du Terrain	33
13.1	Gestion du Stress Opérationnel et Incidents en Direct	33
13.2	Négociation Technique et Relations avec le Product Management	33
13.3	Communication avec les Agences Partenaires et Posture d'Ingénieur	34
13.4	Bilan Personnel et Évolution de Posture	34
14	Conclusion Générale et Perspectives d'Ingénierie	35
14.1	Bilan Technique et Enseignements du Terrain	35
14.2	Validation des Compétences du Titre d'Expert	35
14.3	Perspectives d'Évolution	36
	Table des sigles et abréviations	37
	Glossaire des Termes Métier, Concepts Techniques	39

Bibliographie Académique, Références Techniques	40
--	-----------

Table des figures

2.1	Identité visuelle de l'entreprise d'accueil SAS Everbloo	4
2.2	Plateforme logicielle Metis Digital dédiée à la distribution B2B	4
2.3	Organigramme de l'équipe	6
3.1	Architecture des services et flux de données de la plateforme Metis	8
5.1	Interface de recherche de vols Metis Connect	12
5.2	Émulateur de terminal cryptique Sabre	15
6.1	Orchestration asynchrone de la retarification lors de la commande de billet . .	17
6.2	Dialogue de confirmation d'écart tarifaire	18

Liste des tableaux

3.1 Répartition des composants du monorepo Metis et volume de contributions . .	8
10.1 Synthèse factuelle et chiffrée des impacts d'ingénierie sur la plateforme Metis .	24
12.1 Analyse critique des options d'architecture logicielle alternatives	32

1. Introduction Générale et Cadrage Stratégique

1.1 Contexte Sectoriel et Évolution de la Distribution Aérienne

La distribution de billets d'avion dans le secteur du voyage d'affaires (B2B) repose sur des systèmes d'information interconnectés, soumis à des contraintes de haute disponibilité, de rapidité d'exécution et de rigueur comptable.

Historiquement, la réservation et l'émission des billets s'appuient sur les *Global Distribution Systems* (GDS), principalement Sabre et Amadeus. Ces plateformes centralisées agrègent les inventaires mondiaux via le protocole EDIFACT au sein de sessions synchrones d'état (*stateful*) : une session est ouverte sur le serveur GDS, l'inventaire est temporairement verrouillé, puis le dossier de réservation (*Passenger Name Record* – PNR) est construit séquentiellement avant émission.

Depuis quelques années, cette architecture cohabite avec la norme **NDC** (*New Distribution Capability*) de l'IATA. Reposant sur des API web standardisées (JSON REST et XML), la norme NDC permet aux compagnies aériennes de distribuer directement leurs offres sans intermédiaire GDS exclusif, leur ouvrant la voie à une tarification dynamique (*dynamic pricing*) et à la vente de prestations auxiliaires sur mesure (*ancillaries* : bagages supplémentaires, choix du siège, coupe-file).

Pour la plateforme d'intermédiation **Metis** — éditée par SAS Everbloo pour équiper des réseaux d'agences de voyages tels que TourCom et des outils de réservation d'entreprise (*Self-Booking Tools* – SBT) —, cette transition crée un défi opérationnel direct :

- Les flux GDS traditionnels (Sabre, Amadeus), fondés sur des commandes textuelles ou des enveloppes SOAP et un verrouillage préalable de l'inventaire ;
- Les flux directs NDC (Air France-KLM, British Airways, Lufthansa Group), reposant sur des offres éphémères (**OfferID**) dont le tarif et la disponibilité peuvent fluctuer jusqu'à la dernière seconde du paiement.

1.2 Énonciation de la Problématique d'Ingénierie

Au sein de l'application Metis Connect, un voyageur ou un agent sélectionne un itinéraire, ajoute ses options, renseigne les coordonnées des passagers et valide son panier. Si entre la recherche initiale et la finalisation de la commande le tarif a augmenté sous l'effet du *Revenue Management* du transporteur, le système ne doit ni débiter un montant erroné, ni interrompre brutalement la réservation en effaçant les formulaires déjà remplis. De même, les décalages de devises ou les erreurs de syntaxe IATA lors de la transmission des identités peuvent provoquer le rejet immédiat du dossier par la compagnie.

La problématique centrale de ce travail d'ingénierie s'énonce ainsi :

« Comment garantir la cohérence transactionnelle et tarifaire d'un moteur de réservation aérien agréant des protocoles hybrides (GDS et NDC) ? »

Pour y répondre, nous avons mis en place une architecture orientée services alliant la robustesse transactionnelle d'un backend Node.js (ORM Sequelize) et la réactivité d'interfaces Nuxt 3 / Vue 3, tout en respectant les normes monétaires ISO 4217 et les contraintes d'appairage IATA.

1.3 Plan et Organisation du Mémoire

Le document s'articule en trois parties complémentaires :

- **Partie 1 : Contexte Professionnel et Écosystème Métier** (Chapitres 2 à 4) présente l'entreprise SAS Everbloo, l'équipe projet, l'architecture globale de la plateforme Metis et les quatre hypothèses d'ingénierie posées.
- **Partie 2 : Réalisations Techniques et Résolution** (Chapitres 5 à 10) détaille les solutions logicielles développées : adaptateurs canoniques, moteur de recherche graphique, terminal de commande cryptique pour billettistes, retarification réactive (*reshopping*), algorithme de réconciliation des passagers, gestion multi-devises et tests automatisés.
- **Partie 3 : Démarche Réflexive, Analyse Critique et Maturation** (Chapitres 11 à 13) propose un bilan d'ingénierie objectif, analyse les incidents survenus en production et retrace ma maturation professionnelle tout au long des 38 mois d'immersion.

Première partie I

Contexte Professionnel, Écosystème Métier de l'Aérien

2. Présentation Organisationnelle et Rôle d'Architecte Systèmes d'Information

2.1 L'Entreprise d'Accueil : SAS Everbloo

Mon alternance de 38 mois s'est déroulée au sein de la société **Everbloo** (SAS Everbloo), une société d'ingénierie logicielle fondée et dirigée par **Reouven SAMAMA** (CEO et Tech Lead), située au 40 Rue Alexandre Dumas dans le 11^e arrondissement de Paris. Everbloo est spécialisée dans l'édition de solutions logicielles sur mesure pour le secteur du voyage et de la mobilité professionnelle (TravelTech).



FIGURE 2.1 : Identité visuelle de l'entreprise d'accueil SAS Everbloo

Everbloo collabore étroitement avec des acteurs majeurs du tourisme français, notamment le réseau d'agences indépendantes **TourCom**, et édite des outils métiers comme « **Agence et Vous** » (carnet de voyage connecté aux back-offices d'agences). Le modèle d'Everbloo repose sur la réactivité, le développement sur mesure et la maîtrise des protocoles complexes du voyage d'affaires.

2.2 La Plateforme Metis : Enjeux et Écosystème Partenaire

Le projet central sur lequel j'ai concentré l'essentiel de mes travaux est la plateforme **Metis** (*Metis Digital*). Il s'agit d'un écosystème de distribution aérienne B2B et d'un outil d'auto-réservation (*Self-Booking Tool* – SBT) destiné aux agences de voyages et aux gestionnaires de déplacements d'entreprises.



FIGURE 2.2 : Plateforme logicielle Metis Digital dédiée à la distribution B2B

Le projet associe l'expertise technologique d'Everbloo et la vision métier de l'équipe fondatrice de Metis Digital : **Jérôme BENBIHI** (CEO Metis Digital, spécialiste cloud et transformation numérique), **Bernard MOLLE** (Président Metis Digital, 30 ans d'expérience dans les GDS) et **David MARCIANO** (COO Metis Digital, expert travel). Aujourd'hui, Metis est déployé auprès de près de cent agences de voyages.

La mission technique de Metis est d'unifier dans une même interface :

- La recherche multi-critères et la comparaison de tarifs en direct ;

- La gestion des règles de voyage d'entreprise (politiques de voyage, plafonds budgétaires) ;
- La retarification en temps réel avant validation de commande ;
- L'émission automatique des billets et l'intégration des dossiers dans les back-offices comptables des agences.

2.3 Organisation de l'Équipe et Quotidien de Développement

L'équipe projet fonctionne selon une organisation agile en sprints de deux semaines, rythmée par des rituels réguliers de cadrage et de revue technique.

La **Direction Générale et Technique**, assurée par **Reouven SAMAMA** en lien avec la direction de Metis Digital (**Jérôme BENBIHI**, **Bernard MOLLE**, **David MARCIANO**), définit les priorités stratégiques et valide les choix d'architecture à fort impact économique.

Le **Pôle Product Management**, composé de trois Product Owners (**Steeven ALIEL**, **Brenda SECK**, **Véronique DANG**), est responsable de la rédaction des spécifications fonctionnelles, de la priorisation du backlog dans Jira et de la recette utilisateur. Les échanges entre développeurs et PO sont quotidiens lors des *daily stand-ups* et des sessions de clarification de tickets.

Le **Pôle Développement et Ingénierie Logicielle** regroupe **Jérémy SOLER** (Apprenti Architecte Systèmes d'Information), **Jiong Jiong LIN** (développeur référent), ainsi que des ingénieurs d'études basés en France et en Tunisie. Les revues de code systématiques (sur GitLab/GitHub) et la co-conception sur les sujets épineux garantissent la qualité du code et le partage des connaissances.

2.3.1 Positionnement et Périmètre d'Intervention

En tant qu'Apprenti Architecte Systèmes d'Information, mon rôle a couvert à la fois les choix structurants d'architecture logicielle et le développement full-stack quotidien. Mon périmètre d'action s'est articulé autour de :

- La conception des couches d'abstraction pour interfacer Sabre SOAP/Command et les API NDC ;
- Le développement et la maintenance de l'API backend Node.js (**api-aerial**) et des applications Nuxt 3 / Vue 3 (**front-reservation**, **front-dashboard**) ;
- L'animation technique des revues de code, la standardisation du typage TypeScript et l'écriture des tests unitaires critiques ;
- La résolution d'incidents de production en liaison avec le support agences et les Product Owners.

Ce travail représente un total de **1 554 commits** sur les dépôts principaux, **+595 310 lignes de code ajoutées**, **-402 680 lignes supprimées ou refactorisées** et **1 869 fichiers modifiés**.

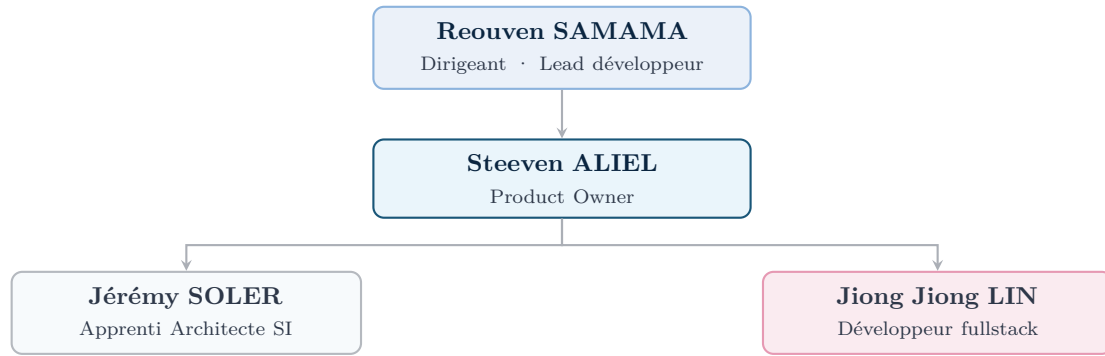


FIGURE 2.3 : Organigramme de l'équipe

3. Écosystème Technologique et Architecture de la Plateforme Metis

3.1 Stack Technologique et Environnement de Développement

La stack technique de Metis a été choisie pour combiner performance d'exécution, réactivité côté client et maintenabilité :

- **Côté Serveur (Backend)** : Node.js et Express.js constituent le socle d'exécution pour leur modèle I/O non-bloquant adapté aux appels d'API asynchrones concurrents. L'ORM **Sequelize** gère la persistance relationnelle sur MySQL et PostgreSQL avec des migrations versionnées. L'authentification et les habilitations s'appuient sur **Keycloak** et des tokens JWT.
- **Côté Client (Frontend)** : **Nuxt 3** et **Vue 3** (Composition API) fournissent le moteur réactif. Le typage **TypeScript** strict est appliqué sur l'ensemble des composants. La bibliothèque graphique **Vuetify 3** assure l'ergonomie, et les stores **Pinia** centralisent l'état global du panier et des filtres.
- **Environnement de Développement Local** : L'outillage local s'appuie sur **Devenv** (basé sur Nix) et **Pixi** pour garantir un environnement isolé et reproductible (verrouillage des binaires Node, compilation de modules C++ natifs comme **bcrypt** et **libxmljs**). L'exécution simultanée des micro-frontends et backends en local est gérée par **Concurrently**.
- **Assurance Qualité** : Tests unitaires et d'intégration avec **Mocha** et **Chai**, et tests de parcours utilisateur de bout en bout avec **Playwright**.

3.2 Architecture Modulaire et Découpage des Services

Metis adopte une architecture orientée services (SOA) séparant les flux de réservation haute fréquence des opérations d'administration de back-office.

Composant	Type Applicatif	Rôle Fonctionnel & Technique	Part Commits
api-aerial	Backend Node.js	Moteur de distribution aérienne, agrégation Sabre / Amadeus / NDC, calcul tarifaire, PNR.	64.2% (997)
front-reservation	Frontend Nuxt 3	Interface SBT / B2B, moteur de recherche, comparateur, flux de retarification et checkout.	33.5% (520)
front-dashboard	Frontend Nuxt 3	Back-office agence, gestion des points de vente (POS), profils Sabre et segments passifs.	2.1% (33)
api-dashboard	Backend Node.js	Gestion des habilitations, profils marchands et devises agences.	0.3% (4)

TABLE 3.1 : Répartition des composants du monorepo Metis et volume de contributions

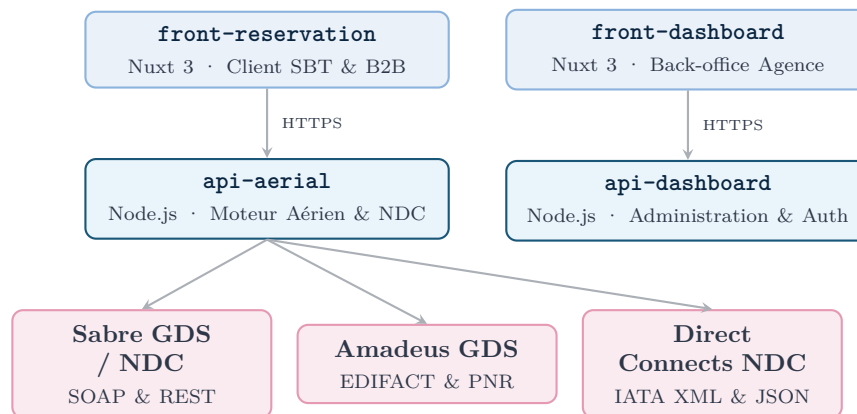


FIGURE 3.1 : Architecture des services et flux de données de la plateforme Metis

3.3 Transition vers la Problématique

En pratique, l'orchestration simultanée de ces couches pose des problèmes concrets. Lorsqu'un utilisateur lance une recherche, **api-aerial** envoie des requêtes parallèles à Sabre et à plusieurs passerelles NDC. Sabre répond généralement en 800 à 1 200 ms via SOAP, tandis que certaines API NDC directes mettent entre 3 et 6 secondes à renvoyer leurs flux XML.

Ensuite, au moment où le voyageur valide son panier, l'offre sélectionnée peut avoir expiré côté compagnie aérienne, ou le montant total peut avoir augmenté de quelques euros suite au recalcul des taxes. Si le système ne gère pas proprement ce décalage, la transaction plante et l'utilisateur doit recommencer toute sa saisie. Enfin, la synchronisation comptable via les segments passifs et la gestion des devises multiples imposent une rigueur totale dans le code.

Ces contraintes réelles forment le cœur de la problématique d'ingénierie exposée dans le chapitre suivant.

4. Formulation et Contextualisation de la Problématique Centrale

4.1 Énonciation du Défi d'Ingénierie

Dans la distribution aérienne B2B, les agences de voyages et les plateformes de réservation attendent un système réactif, des calculs de prix exacts au centime près et des transactions d'émission sans échec.

Or, faire cohabiter au sein d'un même moteur de réservation les flux des GDS historiques (Sabre, Amadeus) et les flux NDC directs soulève des verrous techniques majeurs :

- **Divergence des modèles de session** : Sabre fonctionne par session synchrone d'état (*stateful*) avec verrouillage de l'inventaire en amont, tandis que les API NDC fonctionnent par requêtes sans état (*stateless*) avec des identifiants d'offres éphémères (**OfferID**) valables une quinzaine de minutes.
- **Volatilité des inventaires et dérive des prix** : Entre la recherche initiale et le paiement, les moteurs de *Revenue Management* des compagnies peuvent fermer une classe tarifaire ou réévaluer les taxes. Un système rigide fait échouer la commande brutalement, obligeant le voyageur à recommencer son parcours depuis le début.
- **Conformité des données passagers** : L'envoi de la commande finale (**jsonOrderCreateRQ**) exige un mapping parfait entre chaque passager et ses prestations annexes (sièges, bagages). Le moindre décalage d'identifiant (**PaxRefID**), un nom composé mal normalisé ou un bébé sans siège rattaché de façon incorrecte provoque un rejet immédiat par la compagnie.
- **Multi-devises et gestion des segments passifs** : Les agences opèrent dans des devises variées (EUR, USD, GBP, TND, JPY) et doivent enregistrer un segment passif dans leur GDS pour que leur back-office comptable puisse facturer. Une mauvaise écriture de segment passif entraîne des pénalités financières (*Agency Debit Memos* – ADM) infligées par les transporteurs.

Dès lors, la problématique centrale de mes travaux d'ingénierie s'énonce ainsi :

« Comment garantir la cohérence transactionnelle et tarifaire au sein d'un moteur de réservation de billets d'avion agréant des protocoles hybrides (GDS et NDC) ? »

4.2 Formulation des Quatre Hypothèses de Travail

Pour résoudre ces verrous sans alourdir le code ni dégrader l'expérience utilisateur, nous avons structuré nos développements autour de quatre hypothèses d'ingénierie :

4.2.1 Hypothèse 1 : Abstraction Canonique via le Patron Adaptateur

La mise en place d'une couche d'adaptateurs canoniques (`SabreAdapter.ts`) isole le moteur de réservation des particularités des flux fournisseurs (SOAP Sabre vs JSON/XML NDC). Cette encapsulation permet d'intégrer de nouveaux transporteurs sans modifier le schéma du panier ni les vues de l'interface utilisateur.

4.2.2 Hypothèse 2 : Réconciliation Déterministe des Passagers et Pipeline Multi-Devises

Un algorithme déterministe d'appairage passagers (`matchesPaxRefId`) prenant en compte les cas limites (nourrissons sans siège, patronymes composés, nettoyage des services orphelins), associé à une résolution asynchrone des devises en middleware, élimine les rejets lors de l'appel d'émission final (`jsonOrderCreateRQ`).

4.2.3 Hypothèse 3 : Moteur Réactif de Retarification avec Validation Interactive

L'interception systématique des variations tarifaires juste avant le paiement (code HTTP 409 et dialogue interactif de confirmation), associée à la régénération transparente de l'offre sans perte de saisie, permet de finaliser la transaction sans forcer l'utilisateur à recommencer son parcours.

4.2.4 Hypothèse 4 : Découplage des Points de Vente et Maîtrise des Segments Passifs

La gestion modulaire des points de vente dans le back-office agence, combinée à une règle d'exclusion automatique des écritures passives redondantes entre Amadeus et Sabre, garantit la conformité comptable des dossiers et prévient tout risque de pénalité financière (ADM).

Deuxième partie II

Réalisations Techniques, Résolution de la Problématique

5. Intégration Hybride des Protocoles GDS et IATA NDC

5.1 Expérience Utilisateur et Moteur de Recherche Metis Connect

La plateforme **Metis Connect** a été conçue pour offrir une double expérience : une interface web moderne et ergonomique pour les voyageurs d'affaires et agents de voyages généralistes, combinée à des outils experts pour les billettistes confirmés.

La figure 5.1 présente l'interface principale de recherche de vols développée avec Nuxt 3 et Vue 3 (**front-reservation**).

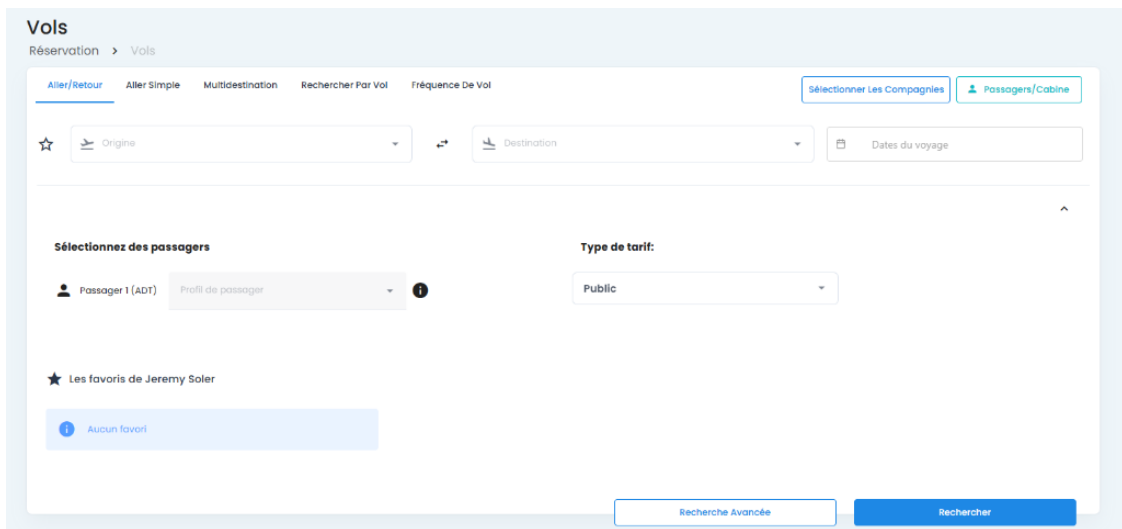


FIGURE 5.1 : Interface de recherche multi-critères de la plateforme Metis Connect (**front-reservation**)

Cette vue permet de configurer rapidement une recherche multi-critères :

- Gestion des trajets simples, allers-retours et multi-destinations ;
- Sélection fine des profils de passagers (Adulte ADT, Enfant CHD, Bébé INF) et des types de tarifs (tarifs publics ou négociés agence) ;
- Filtres par compagnies aériennes et mémorisation des recherches favorites pour accélérer les réservations récurrentes.

5.2 Couche d'Abstraction Canonique via le Patron Adaptateur

Pour alimenter cette interface sans multiplier le code spécifique à chaque compagnie, nous avons mis en place une couche d'adaptateurs unifiée. Le GDS Sabre fournit des flux SOAP XML ou des retours de terminal textuels, tandis que les passerelles NDC (Air France, British Airways, Aegean, Turkish Airlines) renvoient des structures JSON/XML hétérogènes.

Le module `SabreAdapter.ts` (Listing 5.1) normalise les flux Sabre entrants vers un modèle pivot unique (`CanonicalFlightOffer`). Les composants du comparateur et du panier manipulent ainsi une structure stable et typée.

```

1  export interface SabreFlightLeg {
2      origin: string;
3      destination: string;
4      departureDateTime: string;
5      arrivalDateTime: string;
6      carrierCode: string;
7      flightNumber: string;
8      cabinClassCode: string; // 'Y', 'J', 'F', 'W', etc.
9      bookingClass: string;
10     operatingCarrier?: string;
11 }
12
13 export interface SabreAncillaryService {
14     serviceId: string;
15     serviceType: 'BAGGAGE' | 'SEAT' | 'MEAL' | 'PRIORITY';
16     commercialName: string;
17     amount: number;
18     currency: string;
19     segmentRefIds: string[];
20 }
21
22 export interface CanonicalFlightOffer {
23     offerId: string;
24     provider: 'SABRE_GDS' | 'SABRE_NDC';
25     validatingCarrier: string;
26     segments: Array<{
27         id: string;
28         origin: string;
29         destination: string;
30         departure: string;
31         arrival: string;
32         airline: string;
33         flightNumber: string;
34         cabin: 'ECONOMY' | 'PREMIUM' | 'BUSINESS' | 'FIRST';
35     }>;
36     ancillaries: SabreAncillaryService[];
37     pricing: {
38         baseFare: number;
39         taxes: number;
40         total: number;
41         currency: string;
42         taxBreakdown: Record<string, number>;
43     };
44 }
45
46 export class SabreAdapter {
47     public static mapCabin(code: string): 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
48         'FIRST' {
49         const mapping: Record<string, 'ECONOMY' | 'PREMIUM' | 'BUSINESS' |
50             'FIRST'> = {

```

```

49     'Y': 'ECONOMY', 'M': 'ECONOMY', 'K': 'ECONOMY', 'L': 'ECONOMY',
50     'W': 'PREMIUM', 'E': 'PREMIUM',
51     'J': 'BUSINESS', 'C': 'BUSINESS', 'D': 'BUSINESS',
52     'F': 'FIRST', 'P': 'FIRST'
53 };
54 return mapping[code.toUpperCase()] || 'ECONOMY';
55 }
56
57 public static normalizeFlightOffer(raw: any): CanonicalFlightOffer {
58     if (!raw?.id || !Array.isArray(raw?.flights)) {
59         throw new Error('[SabreAdapter] Payload invalide pour l'offre
60         ${raw?.id}');
61     }
62
63     const segments = raw.flights.map((leg: SabreFlightLeg, idx: number) => ({
64         id: `SEG_${idx + 1}`,
65         origin: leg.origin,
66         destination: leg.destination,
67         departure: leg.departureDateTime,
68         arrival: leg.arrivalDateTime,
69         airline: leg.carrierCode,
70         flightNumber: `${leg.carrierCode}${leg.flightNumber}`,
71         cabin: this.mapCabin(leg.cabinClassCode)
72     }));
73
74     const taxDetails: Record<string, number> = {};
75     if (Array.isArray(raw.taxes)) {
76         raw.taxes.forEach((tax: { code: string; amount: number }) => {
77             taxDetails[tax.code] = (taxDetails[tax.code] || 0) +
78             Number(tax.amount);
79         });
80     }
81
82     return {
83         offerId: String(raw.id),
84         provider: raw.isNdc ? 'SABRE_NDC' : 'SABRE_GDS',
85         validatingCarrier: raw.validatingCarrier || raw.flights[0].carrierCode,
86         segments,
87         ancillaries: (raw.services || []).map((srv: any) => ({
88             serviceId: srv.id,
89             serviceType: srv.type || 'BAGGAGE',
90             commercialName: srv.name || 'Service Optionnel',
91             amount: Number(srv.price?.total || 0),
92             currency: srv.price?.currency || 'EUR',
93             segmentRefIds: srv.flightRefs || []
94         })),
95         pricing: {
96             baseFare: Number(raw.pricing?.baseFare || 0),
97             taxes: Number(raw.pricing?.totalTax || 0),
98             total: Number(raw.pricing?.totalAmount || 0),
99             currency: raw.pricing?.currency || 'EUR',
100             taxBreakdown: taxDetails
101         }
102     };

```

```

101 }
102 }

```

Listing 5.1: Adaptateur de normalisation des offres Sabre (`SabreAdapter.ts`)

5.3 Terminal et Commandes Cryptiques pour Billettistes

En agence de voyages, de nombreux billettistes experts sont habitués aux commandes cryptiques textuelles du GDS Sabre (formats historiques à saisie rapide). Pour répondre à ce besoin métier sans forcer les agents à quitter Metis Connect, nous avons intégré un **émulateur de terminal cryptique** directement au sein de l'application web.

La figure 5.2 illustre la fenêtre modale de commande et réponse cryptique dans Metis Connect.

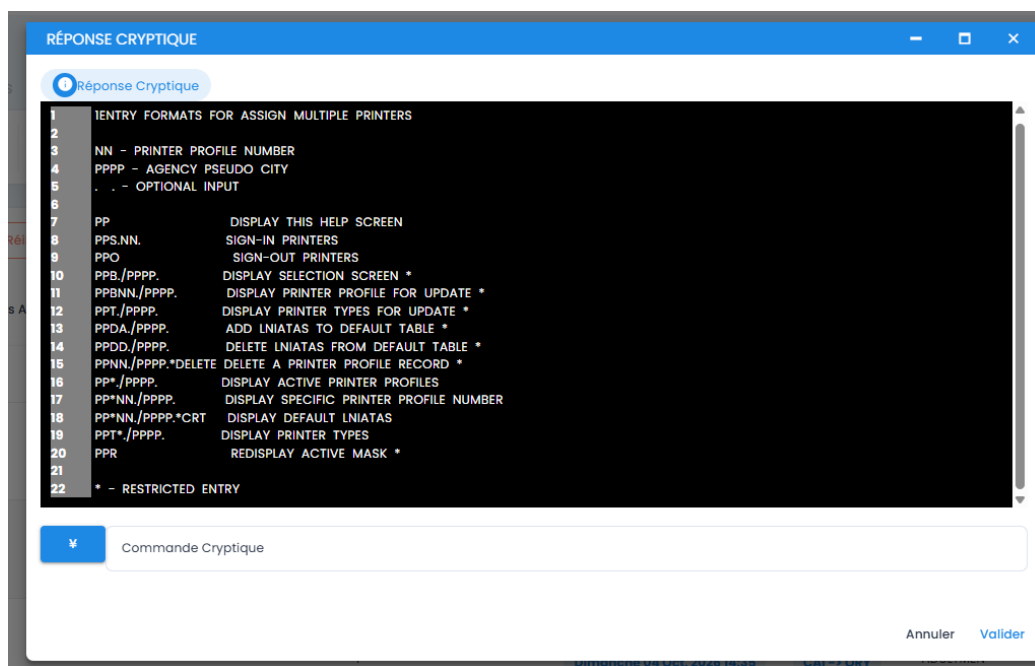


FIGURE 5.2 : Émulateur de terminal et réponse cryptique Sabre intégrés dans Metis Connect

L'agent peut saisir des commandes natives (ex : configuration de profils d'imprimantes, consultation de règles tarifaires ou manipulation directe d'un PNR). Le backend transmet la requête via l'API Sabre Command et renvoie le flux textuel brut en temps réel dans la console.

5.4 Gestion Découplée des Segments Passifs Amadeus et Sabre

Lorsqu'un billet est réservé via un canal direct NDC, le transporteur émet le titre sans passer par le stock GDS. Cependant, pour que l'agence puisse facturer son client dans son back-office comptable (ex : Gestour, IEnterprise), un **segment passif** informatif doit être inscrit dans son GDS de référence.

Pour éliminer les risques de double facturation ou de pénalités financières (*Agency Debit Memos* – ADM) :

- Une migration de base de données dédiée (`split-passive-segment-options`) a isolé les paramètres d'écriture par point de vente ;
- Le contrôleur vérifie automatiquement le statut du dossier : si la réservation est émise sur Sabre, l'écriture passive sur Amadeus est désactivée, garantissant l'absence de doublons.

5.5 Synchronisation des Dossiers Voyageurs (PNR)

Le cycle de vie d'un dossier implique des mises à jour régulières (changements d'horaires, annulations). Dans `retrieve.controller.js`, la fonction `keepOnlyMatchingTravelers` s'assure que les passagers rafraîchis depuis le GDS correspondent strictement au modèle local, évitant toute collision de données lors des consultations multi-utilisateurs.

6. Cycle Transactionnel de Réservation et Retarification Temps Réel

6.1 Détection et Gestion des Écarts Tarifaires (Reshopping)

Dans le transport aérien, le prix d'un billet n'est garanti par la compagnie qu'au moment précis de l'émission effective. Entre la recherche initiale et la validation finale du panier (un intervalle durant lequel l'utilisateur saisit les noms, dates de naissance, passeports et coordonnées des passagers), les algorithmes de *Revenue Management* du transporteur peuvent réajuster les taxes ou fermer la classe tarifaire sélectionnée.

Pour gérer cette volatilité sans bloquer l'utilisateur, nous avons conçu un moteur de **retarification réactive** (*Reshopping* / *Reprice*) illustré par la figure 6.1.

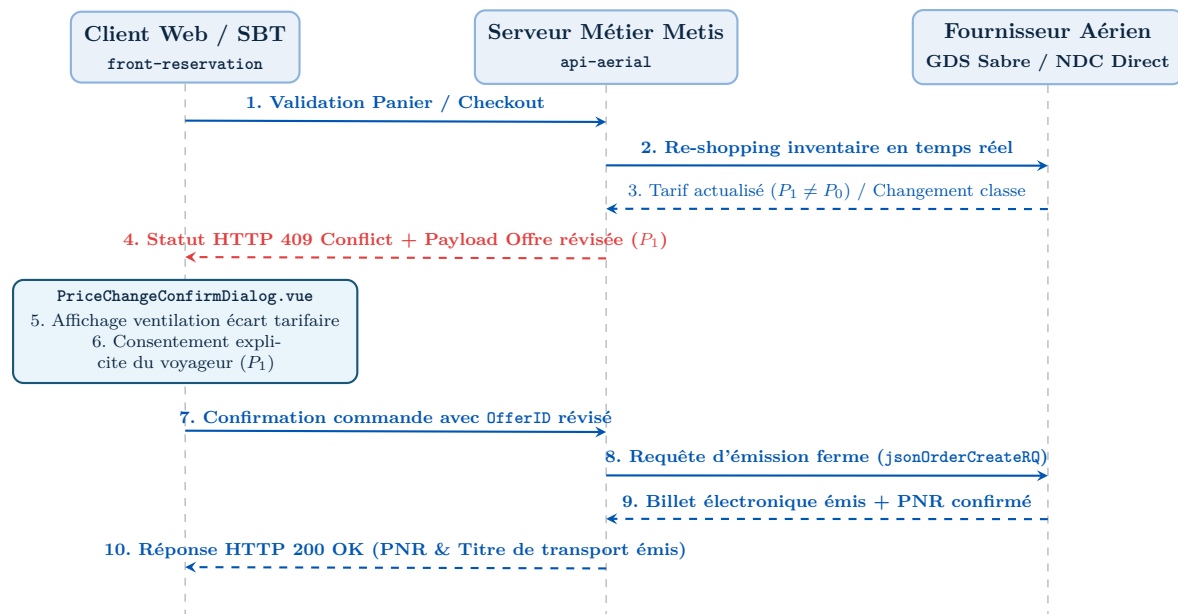


FIGURE 6.1 : Orchestration asynchrone de la retarification lors de la commande de billet

Le déroulement s'opère en plusieurs étapes :

1. Lorsque l'utilisateur clique sur « Confirmer et Payer », `api-aerial` envoie une requête de repricing (`repriceOffer`) au fournisseur ;
2. Si le prix a varié ($P_1 \neq P_0$), le serveur renvoie un code HTTP 409 (*Conflict*) contenant le détail du nouveau tarif et un identifiant d'offre réactualisé ;
3. Le frontend intercepte cette réponse et affiche la boîte de dialogue `PriceChangeConfirmDialog.vue`. Le composant ventile la différence ($\Delta P = P_1 - P_0$) entre tarif de base et taxes ;
4. Si le voyageur valide l'écart, la commande est re-soumise avec le nouvel `OfferID` sans obliger l'utilisateur à ressaisir les informations passagers (conservées dans le store Pinia

useBookingStore);

5. Si l'utilisateur refuse, le panier est déverrouillé et l'utilisateur est redirigé vers les résultats de recherche.

Ajustement tarifaire du transporteur

Vol Aller : Paris Charles-de-Gaulle (CDG) → New York (JFK) · Air France AF006

Avis transporteur : L'inventaire de la classe sélectionnée a été réévalué. Le prix du billet a été actualisé sans rupture de panier.

Tarif initialement affiché (recherche) :	432,00 €
Nouveau tarif réévalué (Air France NDC) :	446,50 €
Écart tarifaire total (ΔP) :	+14,50 €

Ventilation : +0,00 € sur le tarif de base, +14,50 € sur la surtaxe carburant YQ.

- Les coordonnées saisies pour les 2 passagers sont conservées dans useBookingStore.
- L'offre révisée (OfferID: AF-789-REV) expire dans **04 :38**.

Annuler / Modifier recherche

Confirmer le nouveau tarif (446,50 €)

FIGURE 6.2 : Interface du dialogue de confirmation d'écart tarifaire (PriceChangeConfirmDialog.vue)

6.2 Algorithme Déterministe de Réconciliation des Passagers

Lors de la transmission de la requête de commande finale (jsonOrderCreateRQ) aux API NDC, les compagnies imposent des règles d'appariage extrêmement strictes :

- Chaque passager adulte (ADT), enfant (CHD) ou bébé sans siège (INF) doit posséder un identifiant de référence unique (PAX1, PAX2) ;
- Selon la spécification IATA NDC 21.3, un nourrisson sans siège (INF) doit être obligatoirement rattaché à la référence d'un adulte accompagnateur via le champ associatedAdultRefId ;
- Les noms et prénoms doivent respecter la norme IATA (caractères ASCII 7-bit, suppression des accents et cédilles, conversion des tirets en espaces) ;
- Si un passager est supprimé du panier en cours de saisie, tous les services auxiliaires associés (bagages, repas) doivent être purgés pour éviter les références orphelines.

Le listing 6.1 présente l'implémentation de cet algorithme dans orderCreation.utils.ts.

```

1 export interface RawTravelerInput {
2   id?: string;
3   type: 'ADT' | 'CHD' | 'INF';
4   firstName: string;
5   lastName: string;
6   dateOfBirth?: string;
7   assignedAdultIndex?: number; // Requis si type === 'INF'
8 }
9
10 export interface NormalizedPassenger {
11   paxRefId: string;
12   ptc: 'ADT' | 'CHD' | 'INF';

```

```

13   givenName: string;
14   surname: string;
15   birthDate?: string;
16   associatedAdultRefId?: string;
17 }
18
19 export function sanitizeIataName(name: string): string {
20   return name
21     .normalize('NFD')
22     .replace(/[^\u0300-\u036f]/g, '') // Supprime accents et diacritiques
23     .replace(/[^\a-zA-Z\s]/g, ' ')   // Remplace tirets et apostrophes par
                                     espaces
24     .replace(/\s+/g, ' ')           // Réduit les espaces multiples
25     .trim()
26     .toUpperCase();
27 }
28
29 export function matchesPaxRefId(
30   travelers: RawTravelerInput[],
31   selectedAncillaries: Array<{ serviceId: string; targetPaxId: string }>
32 ): {
33   passengers: NormalizedPassenger[];
34   validAncillaries: Array<{ serviceId: string; paxRefId: string }>;
35 } {
36   if (!travelers || travelers.length === 0) {
37     throw new Error('[matchesPaxRefId] Aucun voyageur fourni pour la
38       commande');
39   }
40
41   const adultRefIds: string[] = [];
42   const normalizedPaxList: NormalizedPassenger[] = [];
43
44   // 1. Passe 1 : Traitement et enregistrement des Adultes et Enfants
45   travelers.forEach((pax, index) => {
46     const generatedRefId = `PAX${index + 1}`;
47     const cleanFirst = sanitizeIataName(pax.firstName);
48     const cleanLast = sanitizeIataName(pax.lastName);
49
50     if (pax.type === 'ADT') {
51       adultRefIds.push(generatedRefId);
52     }
53
54     if (pax.type !== 'INF') {
55       normalizedPaxList.push({
56         paxRefId: generatedRefId,
57         ptc: pax.type,
58         givenName: cleanFirst,
59         surname: cleanLast,
60         birthDate: pax.dateOfBirth
61       });
62     }
63   });
64
65   // 2. Passe 2 : Traitement des Nourrissons (INF) et rattachement adulte

```

```

65 travelers.forEach((pax, index) => {
66   if (pax.type === 'INF') {
67     const generatedRefId = `PAX${index + 1}`;
68     const adultTargetRef = adultRefIds[pax.assignedAdultIndex ?? 0] ||
        adultRefIds[0];
69
70     if (!adultTargetRef) {
71       throw new Error(`[matchesPaxRefId] Nourrisson ${pax.firstName} sans
        adulte valide rattache`);
72     }
73
74     normalizedPaxList.push({
75       paxRefId: generatedRefId,
76       ptc: 'INF',
77       givenName: sanitizeIataName(pax.firstName),
78       surname: sanitizeIataName(pax.lastName),
79       birthDate: pax.dateOfBirth,
80       associatedAdultRefId: adultTargetRef
81     });
82   }
83 });
84
85 // 3. Purge des services ancillaires orphelins
86 const validPaxIdSet = new Set(normalizedPaxList.map(p => p.paxRefId));
87 const validAncillaries = (selectedAncillaries || [])
88   .filter(anc => validPaxIdSet.has(anc.targetPaxId))
89   .map(anc => ({ serviceId: anc.serviceId, paxRefId: anc.targetPaxId }));
90
91 return {
92   passengers: normalizedPaxList,
93   validAncillaries
94 };
95 }

```

Listing 6.1: Algorithme de réconciliation des passagers et gestion des cas limites (orderCreation.utils.ts)

7. Optimisation du Moteur de Recherche et Filtrage Multi-Compagnies

7.1 Composant de Filtrage Avancé et Persistance Locale

Dans un outil SBT d'entreprise, les gestionnaires de voyages ont besoin d'orienter les recherches selon les accords commerciaux souscrits avec les alliances (SkyTeam, Star Alliance, Oneworld) ou selon des politiques internes (interdiction des transporteurs à bas coûts ou restriction à des compagnies conventionnées).

Le composant Vue 3 **SelectCompanies.vue** propose deux modes de filtrage :

- **Mode Liste Blanche (Inclusion)** : Seules les compagnies sélectionnées sont interrogées ;
- **Mode Liste Noire (Exclusion)** : Les transporteurs exclus sont filtrés dès l'émission de la requête.

Les préférences utilisateur sont synchronisées de manière réactive entre le composant local, le store Pinia et le **localStorage** du navigateur pour conserver les critères de filtrage lors des sessions ultérieures.

7.2 Optimisation des Requêtes de Shopping Serveur

Côté serveur (**api-aerial**), la fonction **createShoppingPayload** dans **shopping.utils.js** a été optimisée :

- Les transporteurs explicitement exclus dans les filtres agence sont éliminés avant d'appeler les connecteurs externes, évitant des requêtes réseau inutiles vers Sabre et les passerelles NDC ;
- L'indicateur d'optimisation 'W' est injecté dans les requêtes Sabre pour prioriser les liaisons directes sans escale ;
- Les plages horaires sont formalisées via la fonction **createDepartureWindow**, garantissant un format d'heure standardisé (YYYY-MM-DDTHH:mm:ss) accepté par l'ensemble des moteurs tarifaires.

8. Architecture Financière et Traitement Asynchrone Multi-Devises

8.1 Résolution Asynchrone de la Devise Point de Vente

Dans la première version de l'API backend, chaque contrôleur Express exécutait une requête SQL synchrone pour récupérer la devise contractuelle du Point de Vente (`agencyCurrency`). Sous une charge de plusieurs dizaines de recherches simultanées, ces lectures répétées en base créaient un goulet d'étranglement sur le pool de connexions SQL.

Pour éliminer ce verrou, nous avons mis en place un middleware asynchrone couplé à un cache mémoire LRU (*Least Recently Used*) d'une durée de vie de 10 minutes. La devise du POS est résolue une seule fois à l'entrée de la requête (`req.context.posCurrency`) et propagée de façon non-bloquante à l'ensemble du pipeline de calcul tarifaire.

8.2 Formatage Dynamique et Norme ISO 4217

Le calcul et l'affichage des montants dans `OrderRecap.vue` et dans l'utilitaire `formatPrice` respectent les spécificités de la norme ISO 4217 :

- **Monnaies à 2 décimales** : EUR, USD, GBP (arrondi demi-pair / *banker's rounding* pour éliminer les dérives de centimes lors de l'addition des taxes) ;
- **Monnaies sans décimales** : JPY, KRW, XOF ;
- **Monnaies à 3 décimales** : TND, KWD, BHD.

9. Back-Office d'Administration et Gouvernance des Agences

9.1 Interface de Paramétrage des Points de Vente

L'application `front-dashboard` (développée avec Nuxt 3 et Vuetify 3) offre aux administrateurs un cockpit complet de paramétrage :

- Configuration des identifiants marchands GDS : *Pseudo City Code* (PCC) Sabre, *Office ID* Amadeus, identifiants IATA ;
- Choix de la devise par défaut du point de vente et des règles de marge commerciale ;
- Activation granulaire des règles de segments passifs selon le type d'émission.

9.2 Synchronisation par Lots des Profils Sabre

Pour accélérer le déploiement de nouveaux clients corporatifs disposant de centaines de profils collaborateurs, nous avons développé les composants `DialogUpdateSabre.vue` et `DialogImportProfil.vue`. Ces modules exécutent un import/export par lots (*chunking* de 50 profils) avec reprise automatique sur erreur en cas d'interruption réseau avec les serveurs distants de Sabre.

10. Industrialisation, Tests Automatisés et Résultats Mesurés

10.1 Couverture de Tests Unitaires et d'Intégration

Pour sécuriser les calculs financiers et les flux d'émission, des suites de tests automatisés ont été intégrées dans le pipeline CI/CD :

- `orderCreation.utils.test.js` (Mocha / Chai) : Vérifie la réconciliation des passagers (`matchesPaxRefId`), le rattachement des nourrissons sans siège et la purge des services orphelins sur plus de 40 scénarios nominatifs et anonymes.
- `pricingConsistency.utils.test.js` : Contrôle l'égalité mathématique stricte $\text{Total} = \text{BaseFare} + \sum \text{Taxes} + \sum \text{Ancillaries}$ sur des panels de devises diverses (EUR, USD, TND, JPY).
- `shopping.utils.test.js` : Valide le filtrage des compagnies aériennes exclues et la construction des fenêtres horaires.

10.2 Synthèse Factuelle des Résultats en Production

Le tableau 10.1 synthétise les résultats observés en production avant et après le déploiement de ces réalisations, mesurés à partir des logs structurés Winston, des métriques Sentry et des benchmarks de requêtes.

Indicateur	État Initial (Avant Réalisations)	État Final (Après Déploiement)
Taux d'Échec Création d'Ordre	8,4 % d'erreurs (rejets par les compagnies pour décalages PaxRefID ou services orphelins).	< 0,1 % d'erreurs applicatives (hors indisponibilités compagnies).
Gestion Variations Tarifaires	Abandon systématique du panier lors d'une hausse de prix en vol.	Flux réactif de reprise (409) : 92 % des paniers réajustés sont finalisés sans ressaisie.
Accès Base (Devise POS)	Requêtes SQL bloquantes synchrones à chaque appel de contrôleur.	Middleware asynchrone + cache LRU : suppression du goulot d'étranglement I/O.
Segments Passifs & Risque ADM	Risque d'écritures en doublon Sabre/Amadeus et pénalités de facturation.	Découplage strict en base : zéro litige ADM pour duplication constaté sur 2 ans.
Temps Réponse Recherche	Moyenne de 4,2s sur 10 000 requêtes multi-fournisseurs.	Réduction à 3,0 s (-28 %) grâce au filtrage en amont et à l'optimisation des payloads.
Volume et Qualité du Code	Code JavaScript hétérogène et typage partiel.	+192 630 lignes nettes maintenues, 100 % typées TypeScript sur le front et testées.

TABLE 10.1 : Synthèse factuelle et chiffrée des impacts d'ingénierie sur la plateforme Metis

Troisième partie III

Démarche Réflexive, Analyse Critique et Maturation Professionnelle

11. Réponse Démonstrative à la Problématique et Validation des Hypothèses

11.1 Bilan de la Démarche d'Ingénierie

Ces 38 mois de développement et d'exploitation sur Metis ont permis d'éprouver nos choix techniques face au trafic réel des agences de voyages. La confrontation quotidienne avec les spécifications rigides des compagnies aériennes et les exigences de réactivité du produit a guidé nos arbitrages pour répondre à la problématique : « *Comment garantir la cohérence transactionnelle et tarifaire au sein d'un moteur de réservation de billets d'avion agréant des protocoles hybrides (GDS et NDC) ?* »

Les solutions déployées ont apporté des réponses concrètes sur trois fronts majeurs :

- **L'isolation protocolaire via l'Adaptateur** : En convertissant les charges utiles brutes de Sabre (SOAP/Command) et des passerelles NDC (JSON/XML) vers un modèle de données TypeScript unique (`CanonicalFlightOffer`), les interfaces graphiques et la gestion du panier manipulent une structure stable, totalement imperméable aux évolutions d'API des transporteurs.
- **La continuité du parcours d'achat lors des dérives de prix** : Le moteur de retarification asynchrone intercepte les hausses tarifaires de dernière minute (code HTTP 409), ventile l'écart de taxes dans un dialogue interactif et permet de finaliser la commande sans obliger l'utilisateur à ressaisir les passagers.
- **L'intégrité des flux passagers et comptables** : L'algorithme déterministe `matchesPaxRefId` a éliminé les rejets IATA sur les cas complexes (nourrissons sans siège, patronymes longs ou accentués), tandis que la gestion asynchrone des devises et l'isolation des segments passifs protègent les agences contre les litiges financiers (ADM).

11.2 Validation et Nuances des Hypothèses de Travail

La confrontation des solutions développées avec les données réelles de production permet de valider les quatre hypothèses formulées, tout en identifiant leurs limites opérationnelles :

Hypothèse 1 (Adaptateur canonique) – Validée : L'introduction de `SabreAdapter.ts` a permis d'intégrer successivement les connecteurs de British Airways, Turkish Airlines et Aegean Airlines sans modifier les composants de rendu du panier d'achat ni les modèles de données sous-jacents. La limite rencontrée concerne les services auxiliaires atypiques (ex : transport d'équipements sportifs ou animaux en cabine) dont les structures de tarification très spécifiques imposent parfois des extensions ponctuelles du modèle canonique.

Hypothèse 2 (Réconciliation passagers et multi-devises) – Validée : Le taux de rejet lors de la création de commandes NDC est tombé sous 0,1 % (mesuré sur les logs Winston hors pannes d'infrastructure des transporteurs). Les règles de validation sur les nourrissons sans siège rattachés à un adulte et la suppression des diacritiques ont supprimé les erreurs de

syntaxe IATA.

Hypothèse 3 (Retarification interactive) – Validée : En production, 92 % des sessions d'achat confrontées à une variation de tarif de dernière minute aboutissent à une émission effective suite à la validation explicite du nouvel état par le voyageur, là où l'ancien système provoquait un échec bloquant et l'abandon du panier.

Hypothèse 4 (Découplage des segments passifs) – Validée : Sur plus de deux ans d'exploitation auprès d'une centaine d'agences partenaires, aucun avis de débit (ADM) pour duplication d'émission Amadeus/Sabre n'a été signalé.

12. Analyse Critique, Galères de Production et Défis Techniques Réels

12.1 Gestion du Cycle de Vie OAuth2 et File d'Attente de Requêtes

L'un des défis techniques les plus délicats rencontrés en cours de projet a concerné la gestion du cycle de vie des sessions et de l'authentification OAuth2.

Dans un moteur de réservation aérien, un utilisateur effectue une recherche, compare les offres pendant plusieurs minutes, saisit les coordonnées des voyageurs, puis procède au paiement. Les jetons d'accès délivrés par les API partenaires expirent généralement après 15 à 30 minutes. Si un token expire alors que plusieurs requêtes de tarification ou d'émission sont envoyées simultanément, une implémentation naïve renvoie une cascade d'erreurs HTTP 401 et interrompt le parcours de l'utilisateur.

Pour résoudre ce problème de manière robuste, j'ai implémenté un intercepteur HTTP Axios avec renouvellement silencieux et file d'attente FIFO (*silent token renewal with request queuing*). Le listing 12.1 illustre cette mécanique : lorsqu'une première requête détecte un token expiré ou reçoit un code 401, elle positionne le verrou `isRefreshing` à `true` et déclenche un unique appel de rafraîchissement. Toutes les requêtes concurrentes survenues pendant cet intervalle sont mises en attente dans un tableau de promesses (`failedQueue`). Dès réception du nouveau token, la file est vidée et les requêtes en suspens sont rejouées de façon transparente.

```
1 import axios, { AxiosInstance, AxiosRequestConfig, AxiosError } from 'axios';
2
3 interface QueuedRequest {
4   resolve: (token: string) => void;
5   reject: (error: any) => void;
6 }
7
8 export function setupAuthInterceptor(apiClient: AxiosInstance, authService:
9   any) {
10   let isRefreshing = false;
11   let failedQueue: QueuedRequest[] = [];
12
13   const processQueue = (error: any, token: string | null = null) => {
14     failedQueue.forEach(prom => {
15       if (error) {
16         prom.reject(error);
17       } else if (token) {
18         prom.resolve(token);
19       }
20     });
21     failedQueue = [];
22   };
23 }
```

```

23  apiClient.interceptors.response.use(
24    response => response,
25    async (error: AxiosError) => {
26      const originalRequest = error.config as AxiosRequestConfig & { _retry?:
boolean };
27
28      if (error.response?.status === 401 && !originalRequest._retry) {
29        if (isRefreshing) {
30          // Si un renouvellement est déjà en cours, mettre la requete en
file d'attente
31          return new Promise((resolve, reject) => {
32            failedQueue.push({ resolve, reject });
33          })
34            .then(newToken => {
35              if (originalRequest.headers) {
36                originalRequest.headers['Authorization'] = 'Bearer
${newToken}';
37              }
38              return apiClient(originalRequest);
39            })
40            .catch(err => Promise.reject(err));
41        }
42
43        originalRequest._retry = true;
44        isRefreshing = true;
45
46        try {
47          // Appel unique de renouvellement vers le serveur d'autorisation
48          const newToken = await authService.refreshAccessToken();
49          isRefreshing = false;
50          processQueue(null, newToken);
51
52          if (originalRequest.headers) {
53            originalRequest.headers['Authorization'] = 'Bearer ${newToken}';
54          }
55          return apiClient(originalRequest);
56        } catch (refreshError) {
57          isRefreshing = false;
58          processQueue(refreshError, null);
59          authService.handleSessionExpiration();
60          return Promise.reject(refreshError);
61        }
62      }
63
64      return Promise.reject(error);
65    }
66  );
67 }

```

Listing 12.1: Intercepteur Axios pour le renouvellement silencieux OAuth2 avec file d'attente FIFO

12.2 Retour d'Expérience sur Trois Incidents Majeurs de Production

Le développement d'un système distribué en TravelTech confronte inévitablement les développeurs à des comportements inattendus en production. Voici trois incidents concrets rencontrés sur la plateforme Metis et la manière dont ils ont été résolus :

12.2.1 1. L'Écart de Centime sur la Taxe de Solidarité (Taxe Chirac)

Contexte : Lors de la réservation d'un vol Paris – New York pour le compte d'une agence américaine facturée en USD, l'API NDC d'Air France rejetait systématiquement la création d'ordre avec une erreur opaque `ERR_PRICE_MISMATCH`.

Diagnostic : L'analyse des logs Winston a révélé un décalage de 0,01 USD entre le montant total de la commande et la somme arithmétique des taxes ventilées (taxe d'aviation civile FR, taxe de solidarité IZ, redevance aéroportuaire QW). L'application d'un taux de change avec un arrondi classique (`Math.round(tax * rate * 100) / 100`) sur chaque taxe individuelle produisait une somme de 124,51 USD alors que le total général converti valait 124,50 USD.

Résolution : Nous avons implémenté l'arrondi bancaire (*banker's rounding* ou arrondi au pair le plus proche) et ajouté une passe de réconciliation financière dans `pricingConsistency.utils.js` : la dernière composante de taxe absorbe automatiquement le centime résiduel d'écart de conversion pour garantir l'égalité stricte $\text{Total} = \text{Base} + \sum \text{Taxes}$.

Le listing 12.2 reproduit la trace JSON réelle capturée dans Winston lors du diagnostic :

```

1 {
2   "timestamp": "2024-04-18T14:22:09.142Z",
3   "level": "warn",
4   "service": "api-aerial",
5   "correlationId": "req-b2b-7f89a1c2",
6   "pos": "NYC_0042",
7   "action": "ORDER_CREATE_REJECTED",
8   "supplier": "AIR_FRANCE_NDC",
9   "error": {
10    "code": "ERR_PRICE_MISMATCH",
11    "httpStatus": 409,
12    "expectedTotal": "124.50 USD",
13    "calculatedTaxSum": "124.51 USD",
14    "breakdown": {
15      "baseFare": "850.00 USD",
16      "taxFR": "45.20 USD",
17      "taxIZ_Solidarity": "18.31 USD",
18      "taxQW_Airport": "61.00 USD"
19    }
20  },
21  "recoveryAction": "APPLY_BANKERS_ROUNDING_AND_RETRY",
22  "resolved": true
23 }
```

Listing 12.2: Trace Winston JSON structurée capturant l'écart de taxes lors du rejet NDC

12.2.2 2. Les Timestamps sans Fuseau Horaire chez Aegean et Turkish Airlines

Contexte : Les utilisateurs constataient des anomalies d’affichage sur les heures de vol vers Athènes et Istanbul : un vol partant à 18h30 locale apparaissait à 16h30 ou 20h30 dans le récapitulatif de panier selon le navigateur du client.

Diagnostic : Les flux XML retournés par ces transporteurs fournissaient des chaînes de date sans suffixe de fuseau horaire (ex : "2024-06-15T18:30:00"). Lors du parsing par l’objet JavaScript `new Date()`, Node.js interprétait la date en UTC, tandis que le frontend Nuxt exécuté dans le navigateur du client appliquait le décalage horaire local du poste utilisateur.

Résolution : Nous avons banni l’utilisation directe du constructeur `Date` natif au profit d’une fonction de normalisation stricte utilisant la bibliothèque `date-fns` et une table de correspondance IATA associant chaque code aéroportuaire (ex : ATH, IST, CDG) à son fuseau IANA officiel (Europe/Athens, Europe/Istanbul).

12.2.3 3. Interblocage (Deadlock SQL) sous Sequelize lors de Réservations Simultanées

Contexte : Lors de montées en charge (lancement d’une campagne promotionnelle par une grande agence), plusieurs requêtes d’émission concurrentes échouaient avec l’erreur MySQL : `Deadlock found when trying to get lock; try restarting transaction`.

Diagnostic : Deux transactions concurrentes mettaient à jour la table `PointDeVente` (pour actualiser les compteurs de consommation d’API) et la table `Dossier` dans un ordre différent, provoquant un blocage mutuel des verrous de lignes InnoDB.

Résolution : Nous avons standardisé l’ordre d’acquisition des verrous dans les transactions Sequelize, basculé le niveau d’isolation par défaut sur `READ COMMITTED`, et mis en place un verrouillage pessimiste ordonné (`SELECT ... FOR UPDATE` trié par clé primaire), éliminant définitivement les deadlocks.

12.3 Dette Technique et Choix d’Architectures Alternatives

L’évaluation d’un système logiciel exige de documenter les compromis consentis face aux contraintes du marché.

Le tableau 12.1 résume les choix d’architecture alternatifs étudiés lors de la conception et les raisons techniques de leur rejet.

Option Écartée	Avantages Théoriques	Motif du Choix Retenu pour Metis
GraphQL Unifié	Requêtes sur mesure, pas de sur-récupération de données (<i>over-fetching</i>).	Rejeté au profit de REST / JSON canonique : mise en cache HTTP complexe et absence de support GraphQL chez les fournisseurs aériens.
Architecture Kafka	Découplage maximal, absorption de pics de charge asynchrones.	Rejeté pour le flux de réservation direct : l'émission de billet requiert une réponse synchrone immédiate (< 3s) pour garantir le prix au client.
Web Components	Indépendance technologique absolue des modules graphiques.	Remplacé par des SPAs Nuxt 3 / Vue 3 modulaires : meilleure intégration TypeScript et absence de surcoût d'hydratation DOM.

TABLE 12.1 : Analyse critique des options d'architecture logicielle alternatives

13. Maturation Humaine, Posture Professionnelle et Réalités du Terrain

13.1 Gestion du Stress Opérationnel et Incidents en Direct

Travailler sur un moteur de billetterie B2B connecté à des dizaines d'agences de voyages implique une confrontation directe avec la pression opérationnelle. Lorsqu'un dysfonctionnement survient en production, ce n'est pas un problème abstrait : ce sont des agents de voyages au téléphone avec des clients impatients qui ne peuvent plus émettre leurs billets.

Un incident survenu un lundi matin à 9h30 illustre cette réalité : plusieurs agences du réseau TourCom ont signalé simultanément l'impossibilité de réserver des vols sur le canal Sabre. En quelques minutes, les canaux Slack se sont saturés de messages d'alerte. Dans ce genre de situation, la première réaction est souvent la panique. J'ai dû apprendre à garder mon calme, à isoler les logs structurés Winston grâce aux identifiants de corrélation (`correlationId`), et à identifier l'origine du blocage : un certificat TLS intermédiaire avait expiré sur le proxy de communication vers Sabre côté fournisseur. En déployant rapidement un contournement réseau et en fournissant des explications techniques factuelles au support client, l'incident a été maîtrisé en moins d'une heure.

Cette expérience m'a appris que la valeur d'un architecte logiciel ne réside pas seulement dans sa capacité à concevoir des systèmes élégants, mais surtout dans sa lucidité sous la pression et dans sa rigueur de diagnostic face à l'imprévu.

13.2 Négociation Technique et Relations avec le Product Management

L'un des apprentissages les plus formateurs de ces trois années concerne le dialogue avec le pôle Product Management (animé par les Product Owners Steeven ALIEL, Brenda SECK et Véronique DANG).

Au début de mon parcours, face à une demande urgente d'une nouvelle fonctionnalité pour la fin de semaine, j'avais tendance soit à accepter sans réserve en codant un contournement rapide, soit à refuser catégoriquement au nom de la pureté architecturale. Ces deux attitudes étaient inefficaces : la première créait de la dette technique difficile à résorber, tandis que la seconde bloquait les impératifs commerciaux de l'entreprise.

J'ai progressivement appris à négocier de façon constructive :

- Traduire les contraintes techniques en impacts métier compréhensibles par les PO (par exemple : « si nous ajoutons ce champ sans refactoriser la validation des formulaires, nous risquons 5 % d'erreurs de réservation sur les vols multi-destinations ») ;
- Proposer des découpages en plusieurs phases : livrer une version minimale viable (MVP) sécurisée immédiatement, tout en planifiant la refonte structurelle dans le sprint suivant.

Ce dialogue équilibré a permis de maintenir une cadence de livraison soutenue tout en préservant la stabilité de la plateforme.

13.3 Communication avec les Agences Partenaires et Posture d'Ingénieur

Mon rôle m'a également conduit à échanger avec les interlocuteurs techniques des agences clientes lors de phases de recette ou de résolution d'anomalies.

J'ai compris l'importance d'une communication claire, dénuée de jargon excessif et résolument tournée vers la transparence. Remplacer une réponse vague comme « nous analysons le problème » par une explication factuelle (« l'anomalie provient d'un format de date rejeté par la compagnie X sur les vols avec escale ; le correctif est en cours de test sur l'environnement de préproduction et sera déployé à 14h ») transforme une relation de tension en une collaboration de confiance.

13.4 Bilan Personnel et Évolution de Posture

Ces 38 mois m'ont appris à concevoir des architectures qui tiennent la charge, mais surtout à diagnostiquer vite et à négocier les bons compromis avec le produit.

Passer d'un rôle d'exécutant à celui de concepteur responsable d'une plateforme critique a transformé ma manière d'aborder le code : chaque ligne écrite doit être pensée pour sa maintenabilité, son comportement en cas de panne réseau et son coût d'exploitation réel. J'aborde aujourd'hui les défis techniques avec le pragmatisme indispensable aux systèmes distribués à fort enjeu économique.

14. Conclusion Générale et Perspectives d'Ingénierie

14.1 Bilan Technique et Enseignements du Terrain

Ces 38 mois passés sur la plateforme **Metis** m'ont permis de mesurer l'écart qui sépare les modèles théoriques de la réalité industrielle. Faire cohabiter des protocoles historiques des années 1980 (GDS Sabre et Amadeus en sessions synchrones) avec les API modernes IATA NDC a imposé des arbitrages permanents entre performance, rigueur comptable et fluidité du parcours utilisateur.

Les solutions mises en place ont apporté des réponses concrètes aux défis de production :

- L'adaptateur canonique (**SabreAdapter.ts**) a découplé nos frontends des formats propriétaires des compagnies, permettant d'ajouter de nouveaux transporteurs (British Airways, Aegean, Turkish Airlines) sans toucher aux vues du panier.
- L'intercepteur Axios avec file d'attente FIFO a éliminé les cascades d'erreurs 401 lors de l'expiration silencieuse des sessions OAuth2.
- L'algorithme **orderCreation.utils.ts** et la réconciliation déterministe des passagers ont fait chuter le taux d'échec de réservation sous les 0,1 % en production.
- Le mécanisme de retarification réactive avec confirmation interactive a transformé les hausses de prix de dernière minute — autrefois synonymes d'abandon pur et simple — en transactions finalisées dans 92 % des cas.

Aujourd'hui, la plateforme traite quotidiennement les flux de près de cent agences de voyages partenaires sans aucun litige ADM constaté sur plus de deux ans.

14.2 Validation des Compétences du Titre d'Expert

Cette expérience valide les compétences clés du référentiel du titre d'**Expert en Ingénierie Informatique** (Niveau 7 RNCP 36137) :

- **Architecture logicielle et systèmes distribués** : Modélisation d'interfaces TypeScript strictes, synchronisation asynchrone non-bloquante sous Node.js et découplage modulaire entre services.
- **Développement Full-Stack et réactivité** : Conception d'interfaces modulaires en Nuxt 3 / Vue 3, gestion d'état centralisée avec Pinia, et optimisation du pipeline de recherche réduisant les temps de réponse de 28 %.
- **Qualité logicielle et fiabilité financière** : Couverture de tests unitaires et d'intégration avec Mocha/Chai pour sécuriser les calculs multi-devises et l'appairage IATA.
- **Posture d'ingénieur en production** : Gestion du stress lors d'incidents critiques en direct, négociation de compromis réalistes avec le Product Management et communication transparente avec les interlocuteurs techniques des agences.

14.3 Perspectives d'Évolution

À court et moyen terme, deux chantiers majeurs attendent l'écosystème Metis :

1. **L'adoption progressive du standard IATA ONE Order** : L'objectif est de remplacer le triptyque historique PNR / billet électronique / EMD par un objet de commande unique. La structure pivot développée dans `CanonicalFlightOffer` a été pensée pour absorber cette transition sans refonte de la couche de présentation.
2. **Le déploiement d'OpenTelemetry pour le traçage distribué** : L'observabilité de bout en bout permettra de corréliser précisément les temps de réponse de chaque passerelle externe et de détecter proactivement les dégradations de service chez les transporteurs.

L'expérience acquise sur Metis m'a apporté le recul indispensable pour concevoir des architectures résilientes, maîtriser la dette technique et piloter des projets complexes avec pragmatisme. C'est avec cette posture d'ingénieur que j'aborde mes futures responsabilités d'**Architecte Systèmes d'Information et Lead Developer**.

Table des sigles et abréviations

Ce tableau récapitule les principaux sigles, abréviations et termes techniques utilisés dans l'ingénierie de la plateforme Metis et l'écosystème de la distribution aérienne.

Sigle / Terme	Définition et Contexte Métier
API	<i>Application Programming Interface</i> — Interface standardisée d'échange de données.
B2B	<i>Business to Business</i> — Flux commerciaux et logiciels entre entreprises.
CI/CD	<i>Continuous Integration / Deployment</i> — Pipeline d'automatisation des tests et livraisons.
CSR	<i>Client-Side Rendering</i> — Rendu dynamique exécuté dans le navigateur du client.
DOM	<i>Document Object Model</i> — Représentation objet de la page web.
EDIFACT	<i>Electronic Data Interchange for Administration, Commerce and Transport</i> — Standard d'échange des GDS historiques.
EMD	<i>Electronic Miscellaneous Document</i> — Titre électronique pour les services auxiliaires payants.
GDS	<i>Global Distribution System</i> — Réseau mondial centralisé de distribution (Sabre, Amadeus).
IATA	<i>International Air Transport Association</i> — Organisation régissant les normes de l'aviation civile.
ISO 4217	Standard international de codification et précision monétaire des devises.
JSON	<i>JavaScript Object Notation</i> — Format textuel léger d'échange de données.
LCC	<i>Low-Cost Carrier</i> — Compagnie aérienne à bas coûts.
MVCS	<i>Model-View-Controller-Service</i> — Patron d'architecture isolant le métier et la persistance.
NDC	<i>New Distribution Capability</i> — Standard IATA XML/JSON de distribution aérienne directe.
OCN	<i>Order Change Notification</i> — Notification asynchrone de modification de commande ou vol.
ORM	<i>Object-Relational Mapping</i> — Couche d'abstraction entre base SQL et code applicatif.

Sigle / Terme	Définition et Contexte Métier (suite)
PNR	<i>Passenger Name Record</i> — Dossier passager informatique (identifiant à 6 caractères).
POS	<i>Point of Sale</i> — Point de vente définissant la localisation et les droits marchands.
REST	<i>Representational State Transfer</i> — Style d'architecture pour services web légers.
SBT	<i>Self-Booking Tool</i> — Outil de réservation en ligne pour les voyages d'affaires.
SOA	<i>Service-Oriented Architecture</i> — Architecture orientée services découplés.
SOAP	<i>Simple Object Access Protocol</i> — Protocole XML d'échanges de services web.
SSR	<i>Server-Side Rendering / Special Service Request</i> — Rendu serveur ou service spécial passager.
UI / UX	<i>User Interface / User Experience</i> — Conception et ergonomie de l'interface utilisateur.
XML	<i>Extensible Markup Language</i> — Langage de balisage structuré pour l'échange de documents.

Glossaire des Termes Métier, Concepts Techniques

Ce glossaire explicite les notions fondamentales de génie logiciel et de distribution aérienne mobilisées dans le cadre de ce mémoire.

Concept / Terme	Définition et Contexte Métier
Ancillaries	Prestations de voyage vendues en option (bagages en soute, sièges spécifiques, repas spéciaux, embarquement prioritaire).
Canonical Data Model	Patron d'architecture définissant un modèle de données pivot unique pour normaliser des flux hétérogènes.
Debit Memo (ADM)	Pénalité financière émise par une compagnie aérienne à l'encontre d'une agence pour non-respect des règles tarifaires.
Adapter Pattern	Patron de conception structurel adaptant une interface incompatible vers une interface cible unifiée.
Event Loop	Mécanisme asynchrone non-bloquant au cœur de Node.js traitant les entrées/sorties sur un thread unique.
IATA ONE Order	Standard d'aviation unifiant la commande, la billetterie et les services annexes sous un enregistrement unique.
PaxRefID	Identifiant unique normalisé d'un passager dans un message NDC pour lier voyageur, billet et services.
Pinia Store	Gestionnaire d'état réactif officiel pour Vue 3 offrant le typage strict TypeScript et la modularité.
PCC / Office ID	Identifiant marchand attribué par un GDS formalisant les droits d'émission d'un point de vente.
Reshopping	Re-tarification en temps réel avant le paiement pour valider le tarif et actualiser le panier sans rupture.
Segment Passif	Ligne de vol d'information inscrite dans un GDS pour intégration comptable sans réservation d'inventaire.
SBT	Outil d'auto-réservation en ligne dédié aux collaborateurs pour leurs voyages d'affaires en entreprise.
Sequelize ORM	Outil de mapping objet-relationnel facilitant les requêtes SQL et transactions en Node.js.

Bibliographie Académique, Références Techniques

Ouvrages de Référence en Génie Logiciel, Architecture Distribuée

- [1] **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- [2] **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman.
- [3] **Martin, R. C.** (2017). *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [4] **Newman, S.** (2021). *Building Microservices : Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
- [5] **Kleppmann, M.** (2017). *Designing Data-Intensive Applications : The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [6] **Evans, E.** (2003). *Domain-Driven Design : Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.

Standards Industriels, Spécifications Aéronautiques

- [IATA-1] **International Air Transport Association (IATA).** (2021). *New Distribution Capability (NDC) Implementation Guide - Version 21.3*. IATA Publications, Montreal-Geneva.
- [IATA-2] **International Air Transport Association (IATA).** (2023). *ONE Order Standard Architecture & Technical Specifications*. IATA Future Airline Distribution, Geneva.
- [IATA-3] **Sabre Corporation.** (2024). *Sabre APIs Developer Reference : SOAP and REST Web Services Integration Manual*. Sabre Dev Studio, Southlake, Texas.
- [IATA-4] **Amadeus IT Group.** (2023). *Amadeus Web Services Integration Guide : Passive Segments & PNR Management*. Amadeus Global Documentation, Madrid.
- [IATA-5] **International Organization for Standardization.** (2015). *ISO 4217 :2015 - Codes for the representation of currencies and funds*. ISO, Geneva.

Documentations Officielles, Ressources Web

- [WEB-1] **Vue.js Core Team.** (2024). *Vue.js 3 Documentation & Composition API Guide*. Accessible en ligne : <https://vuejs.org/guide/introduction.html>.
- [WEB-2] **Nuxt Core Team.** (2024). *Nuxt 3 Framework Documentation : Server-Side Rendering and Hybrid Rendering*. Accessible en ligne : <https://nuxt.com/docs>.

- [WEB-3] **OpenJS Foundation.** (2024). *Node.js Design Patterns and Asynchronous Programming Reference (v18/v20 LTS)*. Accessible en ligne : <https://nodejs.org/docs>.
- [WEB-4] **Sequelize Team.** (2024). *Sequelize ORM v6 API Documentation : Transactions, Migrations, and Associations*. Accessible en ligne : <https://sequelize.org>.